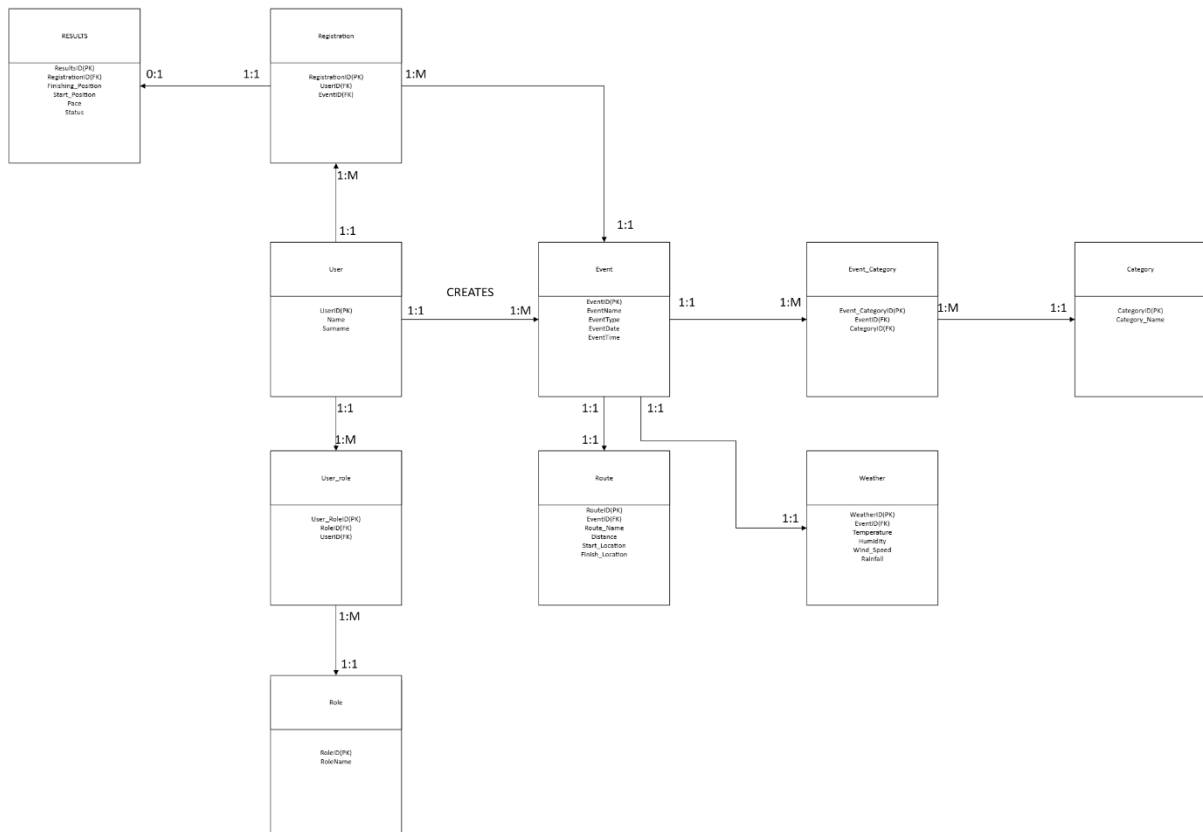


# PROG6212 POE PART 1

THUTO SEHLAPELO

ST10495618

# SECTION A – ENTITY RELATIONSHIP DIAGRAM



## SECTION B – API ENDPOINT PLAN

| HT<br>TP<br>Met<br>hod | Route                   | Descripti<br>on                                            | Role<br>required  | Request body                                             | Expecte<br>d<br>response                                             |
|------------------------|-------------------------|------------------------------------------------------------|-------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| PO<br>ST               | /api/auth/regist<br>er  | Register a<br>new<br>participan<br>t/organise<br>r account | Public            | {FirstName,LastName,email,password<br>,role,phone}       | 201 user<br>created,<br>validatio<br>n error<br>on 400               |
| PO<br>ST               | /api/auth/log<br>in     | Authenti<br>cate user<br>and<br>issue<br>JWT               | Public            | {email, password}                                        | 200<br>token +<br>user<br>summa<br>ry;<br>Invalid<br>credent<br>ials |
| GE<br>T                | /api/users/m<br>e       | Get<br>logged in<br>user's<br>profile                      | Authenti<br>cated | None                                                     | 200<br>profile                                                       |
| PU<br>T                | /api/users/m<br>e       | Update<br>logged in<br>user's<br>profile                   | Authenti<br>cated | {firstname,LastName,Phone}                               | 200<br>update<br>d<br>profile                                        |
| DE<br>LET<br>E         | /api/users/m<br>e       | Deactiva<br>te/delete<br>account                           | Authenti<br>cated | None                                                     | 204                                                                  |
| GE<br>T                | /api/categori<br>es     | List all<br>event<br>categor<br>ies                        | Public            | None                                                     | 200<br>categor<br>y array                                            |
| GE<br>T                | /api/categori<br>es/:id | Get one<br>category                                        | Public            | None                                                     | 200<br>categor<br>y                                                  |
| PO<br>ST               | /api/categori<br>es     | Create<br>an event<br>category                             | Organis<br>er     | {Name,description}                                       | 201<br>categor<br>y                                                  |
| PU<br>T                | /api/categori<br>es/:id | Update a<br>category                                       | Organis<br>er     | {Name,description}                                       | 200<br>categor<br>y                                                  |
| DE<br>LET<br>E         | /api/categori<br>es/:id | Delete a<br>category                                       | Organis<br>er     | None                                                     | 204                                                                  |
| GE<br>T                | /api/events             | Search<br>events                                           | Public            | Query params such as category, date,<br>location, status | 200<br>event<br>array                                                |

|        |                            |                                        |               |                                                                           |                    |
|--------|----------------------------|----------------------------------------|---------------|---------------------------------------------------------------------------|--------------------|
| GET    | /api/events/:id            | View event details                     | public        | None                                                                      | 200 event details  |
| POST   | /api/events                | Create an event                        | Organiser     | {categoryId,name.description,eventDate,location,entryFee,maxParticipants} | 201 event          |
| PUT    | /api/events/:id            | Update an event                        | Organiser     | Event fields to update                                                    | 200 update event   |
| DELETE | /api/events/:id            | Cancel/delete an event                 | Organiser     | None                                                                      | 204                |
| GET    | /api/events/:id/enrolments | View participants enrolled in an event | Organiser     | None                                                                      | 200 enrolment list |
| POST   | /api/events/:id/enrolments | Enrol the logged in participant        | Participant   | Optinal {}/Registration details                                           | 201 enrolment      |
| GET    | /api/enrolments/me         | View current user's event enrolments   | Participant   | Optional status/date query params                                         | 200 enrolment list |
| GET    | /api/enrolments/:id        | View an enrolment                      | Authenticated | None                                                                      | 200 enrolment      |
| DELETE | /api/enrolments/:id        | Cancel an enrolment                    | Participant   | None                                                                      | 204                |
| GET    | /api/events/:id/results    | View results for an event              | Public        | None                                                                      | 200 result list    |
| POST   | /api/events/:id/results    | Record a participants result           | organiser     | {participantId,position,FinishTime,Pace}                                  | 201 result         |
| Put    | /api/results/:id           | Correct/update a result                | Organiser     | Results field                                                             | 200 result         |
| DELETE | /api/results/:id           | Remove an incorrect result             | Organiser     | None                                                                      | 204                |

|      |                         |                                             |                          |                                                             |                    |
|------|-------------------------|---------------------------------------------|--------------------------|-------------------------------------------------------------|--------------------|
| GET  | /api/users/me/results   | View personal performance history           | Participant              | Optional event/date/category filters                        | 200 result history |
| GET  | /api/events/:id/route   | Get route information for an event          | Public                   | None                                                        | 200 route          |
| POST | /api/events/:id/route   | Add/update route information                | Organiser                | {distanceKm,routeUrl,description}201/200 route              | 201/200 route      |
| Get  | /api/events/:id/weather | Get eather information for race preparation | Public                   | Optional forecast date                                      | 200 weather r data |
| POST | /api/events/:id/weather | Store/update event weather forecast         | Organiser/system service | {forecastDate,temperature,humidity,windKph,rainProbability} | 201/200 weather    |
| GET  | /api/health             | Check API/service health                    | Public                   | None                                                        | 200 health status  |

### Core HTTP Status Codes

- 200 OK — successful GET/PUT operation
- 201 Created — successful registration/creation
- 204 No Content — successful deletion/cancellation
- 400 Bad Request — invalid request data
- 401 Unauthorized — missing/invalid authentication
- 403 Forbidden — authenticated but insufficient permissions
- 404 Not Found — resource does not exist
- 409 Conflict — duplicate registration or conflicting operation
- 500 Internal Server Error — unexpected server error

## SECTION C – SQL DATABASE SCRIPT

1. CREATE DATABASE RaceDay

```
2. CREATE TABLE Event(  
EventID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
EventName VARCHAR(255) NOT NULL,  
EventType VARCHAR(255) NOT NULL,  
EventDate Date NOT NULL,  
EventTime Time NOT NULL  
);
```

```
3. CREATE TABLE Users(  
UsersID INT IDENTITY(1,1) PRIMARY KEY,  
Name VARCHAR(255) NOT NULL,  
Surname VARCHAR(255) NOT NULL  
);
```

```
4. CREATE TABLE Roles(  
RoleID INT IDENTITY(1,1) PRIMARY KEY,  
RoleName VARCHAR(255) NOT NULL UNIQUE  
);
```

```
5. CREATE TABLE UserRole(  
User_RoleID INT IDENTITY(1,1) PRIMARY KEY,  
UsersID INT NOT NULL,  
RoleID INT NOT NULL,  
CONSTRAINT FK_UserRole_Users  
FOREIGN KEY (UsersID) REFERENCES Users(UsersID),  
CONSTRAINT FK_UserRole_Roles  
FOREIGN KEY (RoleID) REFERENCES Roles(RoleID),  
CONSTRAINT UQ_UserRole  
UNIQUE (UsersID, RoleID)  
);
```

```
6. CREATE TABLE Registration(  

```

```
RegistrationID INT IDENTITY(1,1) PRIMARY KEY,  
UsersID INT NOT NULL,  
EventID INT NOT NULL,  
CONSTRAINT FK_Registration_Users  
FOREIGN KEY (UsersID) REFERENCES Users(UsersID),  
CONSTRAINT FK_Registration_Event  
FOREIGN KEY (EventID) REFERENCES Event(EventID),  
CONSTRAINT UQ_Registration  
UNIQUE (UsersID, EventID)  
);
```

```
7. CREATE TABLE Results(  
ResultsID INT IDENTITY(1,1) PRIMARY KEY,  
FinishingPosition INT,  
StartPosition INT,  
Pace INT,  
Status Varchar(20) NOT NULL,  
RegistrationID INT NOT NULL,  
CONSTRAINT FK_Results_Registration  
FOREIGN KEY (RegistrationID) REFERENCES Registration(RegistrationID)  
);
```

```
8. CREATE TABLE Route(  
RouteID INT IDENTITY(1,1) PRIMARY KEY,  
RouteName VARCHAR(255),  
Distance DECIMAL(6,2),  
StartLocation VARCHAR(255),  
FinishLocation VARCHAR(255),  
EventID INT NOT NULL,  
CONSTRAINT FK_Route_Event  
FOREIGN KEY (EventID) REFERENCES Event(EventID)
```

);

```
9. CREATE TABLE Weather(  
WeatherID INT IDENTITY(1,1) PRIMARY KEY,  
Temperature DECIMAL(5,2),  
Humidity INT,  
WindSpeed DECIMAL(6,2),  
Rainfall INT,  
EventID INT NOT NULL,  
CONSTRAINT FK_Weather_Event  
FOREIGN KEY (EventID) REFERENCES Event(EventID)  
);
```

```
10. CREATE TABLE Category(  
CategoryID INT IDENTITY(1,1) PRIMARY KEY,  
CategoryName VARCHAR(255) NOT NULL UNIQUE  
);
```

```
11. CREATE TABLE EventCategory(  
EventCategoryID INT IDENTITY(1,1) PRIMARY KEY,  
EventID INT NOT NULL,  
CategoryID INT NOT NULL,  
CONSTRAINT FK_EventCategory_Event  
FOREIGN KEY (EventID) REFERENCES Event(EventID),  
CONSTRAINT FK_EventCategory_Category  
FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),  
CONSTRAINT UQ_EventCategory  
UNIQUE (EventID, CategoryID)  
);
```

## POPULATING THE DATABASE

```
INSERT INTO Roles (RoleName)
```



```
VALUES ('Organiser'), ('Participant');
```

```
SELECT * FROM Roles;
```

```
INSERT INTO Users (Name, Surname)
```

```
VALUES ('Thabo', 'Mokoena'),
```

```
('Lerato', 'Dlamini'),
```

```
('Sipho', 'Nkosi'), ('Amahle', 'Zulu');
```

```
SELECT * FROM Users;
```

```
INSERT INTO UserRole (UsersID, RoleID)
```

```
VALUES (1, 1), (2, 1), (3, 2), (4, 2)
```

```
SELECT * FROM UserRole
```

```
INSERT INTO Event ( EventName, EventType, EventDate, EventTime )
```

```
VALUES ( 'Johannesburg City Marathon', 'Marathon', '2026-10-18', '07:00:00' ),
```

```
( 'Soweto Fun Run', 'Fun Run', '2026-11-08', '08:00:00' ),
```

```
( 'Pretoria Charity Race', 'Charity Run', '2026-12-06', '07:30:00' );
```

```
SELECT * FROM Event;
```

```
INSERT INTO Category (CategoryName)
```

```
VALUES ('Road Running'),
```

```
('Marathon'),
```

```
('Competitive'),
```

```
('Fun Run'),
```

```
('Community'),
```

```
('Family'),
```

```
('Charity'),
```

```
('Fitness'),
```

```
('Fundraising');
```

```
SELECT * FROM Category;
```

```
INSERT INTO EventCategory ( EventID, CategoryID )
```

```
VALUES (1, 1),
```

```
(1, 2),
```

```
(1, 3);
```

```
INSERT INTO EventCategory ( EventID, CategoryID )
```

```
VALUES (2, 4),
```

```
(2, 5),
```

```
(2, 6);
```

```
INSERT INTO EventCategory ( EventID, CategoryID )
```

```
VALUES (3, 7),
```

```
(3, 8),
```

```
(3, 9);
```

```
SELECT * FROM EventCategory;
```

```
INSERT INTO Registration ( UserID, EventID ) VALUES (3, 1),
```

```
(4, 1),
```

```
(3, 2),
```

```
(4, 2),
```

```
(3, 3),
```

```
(4, 3);
```

```
SELECT * FROM Registration;
```

```
INSERT INTO Results ( FinishingPosition, StartPosition, Pace, Status, RegistrationID )
```

```
VALUES (1, 1, 5, 'Finished', 1),
```

```
(2, 2, 6, 'Finished', 2),
```

```
(1, 1, 4, 'Finished', 3),
```

```
(2, 2, 5, 'Finished', 4),
```

```
(3, 1, 6, 'Finished', 5),
```

```
(4, 2, 7, 'Finished', 6);
```

```
SELECT * FROM Results;
```

```

INSERT INTO Route ( RouteName, Distance, StartLocation, FinishLocation, EventID )
VALUES ( 'Johannesburg Marathon Route', 42.20, 'Sandton', 'Johannesburg Stadium', 1 ),
( 'Soweto Fun Run Route', 10.00, 'Soweto Stadium', 'Vilakazi Street', 2 ),
( 'Pretoria Charity Route', 21.10, 'Pretoria CBD', 'Union Buildings', 3 );

SELECT * FROM Route;

```

```

INSERT INTO Weather ( Temperature, Humidity, WindSpeed, Rainfall, EventID )
VALUES ( 19.50, 55, 12.50, 0, 1 ),
( 22.00, 60, 10.20, 0, 2 ),
( 18.00, 50, 8.50, 2, 3 );

SELECT * FROM Weather;

```

**SELECT \* FROM Users;**

The screenshot shows the SQL Server Enterprise Manager interface. The central pane displays the query: `SELECT * FROM Users;`. The results pane below shows a table with 4 rows and 3 columns: UserID, Name, and Surname. The status bar at the bottom indicates the query was executed successfully on 9/4/2026 at 6:44 PM.

| UserID | Name   | Surname |
|--------|--------|---------|
| 1      | Tshabo | Mokone  |
| 2      | Lerato | Dlamini |
| 3      | Sipho  | Nkosi   |
| 4      | Amahle | Zulu    |

## SELECT \* FROM Roles;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL script:

```
1 SELECT * FROM Users;  
2  
3 SELECT * FROM Roles;  
4  
5 SELECT * FROM Event;  
6  
7 SELECT * FROM Category;  
8  
9 SELECT * FROM EventCategory;  
10  
11 SELECT * FROM Registration;
```

The query execution results are displayed in the Results pane, showing two rows of data:

| RoleID | RoleName    |
|--------|-------------|
| 1      | Organiser   |
| 2      | Participant |

The status bar indicates "Query executed successfully." and "LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 2 rows".

## SELECT \* FROM Event;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL script:

```
1 SELECT * FROM Users;  
2  
3 SELECT * FROM Roles;  
4  
5 SELECT * FROM Event;  
6  
7 SELECT * FROM Category;  
8  
9 SELECT * FROM EventCategory;  
10  
11 SELECT * FROM Registration;
```

The query execution results are displayed in the Results pane, showing three rows of data:

| EventID | EventName                  | EventType   | EventDate  | EventTime        |
|---------|----------------------------|-------------|------------|------------------|
| 1       | Johannesburg City Marathon | Marathon    | 2026-10-18 | 07:00:00.0000000 |
| 2       | Soweto Fun Run             | Fun Run     | 2026-11-08 | 08:00:00.0000000 |
| 3       | Pretoria Charity Race      | Charity Run | 2026-12-06 | 07:30:00.0000000 |

The status bar indicates "Query executed successfully." and "LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 3 rows".

## SELECT \* FROM Category;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL script:

```
1 SELECT * FROM Users;
2
3 SELECT * FROM Roles;
4
5 SELECT * FROM Event;
6
7 SELECT * FROM Category;
8
9 SELECT * FROM EventCategory;
10
11 SELECT * FROM Registration;
```

The query results are displayed in a table with the following data:

| CategoryID | CategoryName |
|------------|--------------|
| 1          | Charity      |
| 2          | Community    |
| 3          | Competitive  |
| 4          | Family       |
| 5          | Fitness      |
| 6          | Fun Run      |
| 7          | Fundraising  |
| 8          | Marathon     |
| 9          | Road Running |

The status bar indicates "Query executed successfully." and "Row: 1, Col: 1 9 rows".

## SELECT \* FROM EventCategory;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL script:

```
1 SELECT * FROM Users;
2
3 SELECT * FROM Roles;
4
5 SELECT * FROM Event;
6
7 SELECT * FROM Category;
8
9 SELECT * FROM EventCategory;
10
11 SELECT * FROM Registration;
```

The query results are displayed in a table with the following data:

| EventCategoryID | EventID | CategoryID |
|-----------------|---------|------------|
| 1               | 1       | 1          |
| 2               | 2       | 1          |
| 3               | 3       | 1          |
| 4               | 4       | 2          |
| 5               | 5       | 2          |
| 6               | 6       | 2          |
| 7               | 7       | 3          |
| 8               | 8       | 3          |
| 9               | 9       | 3          |

The status bar indicates "Query executed successfully." and "Row: 1, Col: 1 9 rows".

## SELECT \* FROM Registration;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL statement:

```
SELECT * FROM Users;  
SELECT * FROM Roles;  
SELECT * FROM Event;  
SELECT * FROM Category;  
SELECT * FROM EventCategory;  
SELECT * FROM Registration;
```

The query has been executed successfully, and the results are displayed in the Results pane. The results table has 6 rows and 4 columns: RegistrationID, UserID, EventID, and EventCategoryID.

| RegistrationID | UserID | EventID | EventCategoryID |
|----------------|--------|---------|-----------------|
| 1              | 3      | 1       | 3               |
| 2              | 3      | 2       | 3               |
| 3              | 5      | 3       | 3               |
| 4              | 2      | 4       | 1               |
| 5              | 4      | 4       | 2               |
| 6              | 6      | 4       | 3               |

The status bar at the bottom indicates: "Query executed successfully. LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 6 rows".

## SELECT \* FROM Results;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL statement:

```
SELECT * FROM Category;  
SELECT * FROM EventCategory;  
SELECT * FROM Registration;  
SELECT * FROM Results;  
SELECT * FROM Route;
```

The query has been executed successfully, and the results are displayed in the Results pane. The results table has 6 rows and 6 columns: ResultID, FinishingPosition, StartPosition, Pace, Status, and RegistrationID.

| ResultID | FinishingPosition | StartPosition | Pace | Status   | RegistrationID |
|----------|-------------------|---------------|------|----------|----------------|
| 1        | 1                 | 1             | 5    | Finished | 1              |
| 2        | 2                 | 2             | 6    | Finished | 2              |
| 3        | 3                 | 1             | 4    | Finished | 3              |
| 4        | 4                 | 2             | 5    | Finished | 4              |
| 5        | 3                 | 1             | 6    | Finished | 5              |
| 6        | 4                 | 2             | 7    | Finished | 6              |

The status bar at the bottom indicates: "Query executed successfully. LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 6 rows".

## SELECT \* FROM Route;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL code:

```
6 SELECT * FROM Category;  
7  
8 SELECT * FROM EventCategory;  
9  
10  
11 SELECT * FROM Registration;  
12  
13  
14 SELECT * FROM Results;  
15  
16 SELECT * FROM Route;
```

The query was executed successfully, returning 3 rows. The results are displayed in a table with the following columns: RouteID, RouteName, Distance, StartLocation, FinishLocation, and EventID.

| RouteID | RouteName                   | Distance | StartLocation  | FinishLocation       | EventID |
|---------|-----------------------------|----------|----------------|----------------------|---------|
| 1       | Johannesburg Marathon Route | 42.20    | Sandton        | Johannesburg Stadium | 1       |
| 2       | Soweto Fun Run Route        | 10.00    | Soweto Stadium | Vilakazi Street      | 2       |
| 3       | Pretoria Charity Route      | 21.10    | Pretoria CBD   | Union Buildings      | 3       |

The status bar at the bottom indicates: "Query executed successfully. LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 3 rows".

## SELECT \* FROM Weather;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL code:

```
9 SELECT * FROM EventCategory;  
10  
11 SELECT * FROM Registration;  
12  
13 SELECT * FROM Results;  
14  
15 SELECT * FROM Route;  
16  
17 SELECT * FROM Weather;
```

The query was executed successfully, returning 3 rows. The results are displayed in a table with the following columns: WeatherID, Temperature, Humidity, WindSpeed, Rainfall, and EventID.

| WeatherID | Temperature | Humidity | WindSpeed | Rainfall | EventID |
|-----------|-------------|----------|-----------|----------|---------|
| 1         | 19.50       | 55       | 12.50     | 0        | 1       |
| 2         | 22.00       | 60       | 10.20     | 0        | 2       |
| 3         | 18.00       | 50       | 8.50      | 2        | 3       |

The status bar at the bottom indicates: "Query executed successfully. LabVM2049939\SQLEXPRESS (17... LabVM2049939\Student (94) RaceDayTODAY 00:00:00 Row: 1, Col: 1 3 rows".